CSI Intrusion Detection Project Complete Environment, Package, and Jupyter Kernel Setup Guide

Rajshekhar Bhatta

May 13, 2026

Contents

1 Project Overview **3**

2 Project Architecture **3**

3 Linux / Ubuntu Environment Setup **3**
- 3.1 Step 1: Create Project Directory . . . 3
- 3.2 Step 2: Create Python Virtual Environment . . . 3
- 3.3 Step 3: Activate the Environment . . . 4
- 3.4 Step 4: Upgrade pip . . . 4
- 3.5 Step 5: Install Required Packages . . . 4
  - 3.5.1 Core Scientific Packages . . . 4
  - 3.5.2 Machine Learning Packages . . . 4
  - 3.5.3 Serial Communication Packages . . . 4
  - 3.5.4 Notebook and Kernel Packages . . . 4
  - 3.5.5 Visualization Packages . . . 4
  - 3.5.6 Install All Packages Together . . . 4

4 Windows Environment Setup **5**
- 4.1 Step 1: Create Environment . . . 5
- 4.2 Step 2: Activate Environment . . . 5
- 4.3 Step 3: Install Packages . . . 5
- 4.4 Step 4: Register Jupyter Kernel . . . 5

5 Registering the Jupyter Kernel **5**
- 5.1 Linux Kernel Registration . . . 5
- 5.2 Verify Kernel Installation . . . 6

6 Opening the Project in VS Code **6**
- 6.1 Open VS Code . . . 6
- 6.2 Open Notebook . . . 6
- 6.3 Select Kernel . . . 6
- 6.4 If Kernel Does Not Appear . . . 6

7 Testing the Environment **7**

8 TensorFlow GPU Verification **7**
8.1 Check NVIDIA GPU 7
8.2 Check TensorFlow GPU Access 7

9 Package Summary **7**

10 Running the Notebook **8**
10.1 Launch Jupyter Notebook 8

11 Real-Time CSI Dashboard Execution **8**
11.1 Activate Environment 8
11.2 Run Live Dashboard 8

12 Common Problems and Fixes **8**
12.1 Problem: No module named numpy 8
12.2 Problem: Python (csi_env) does not appear 8
12.3 Problem: No module named serial 9
12.4 Problem: TensorFlow GPU not detected 9

13 Final Recommended Kernel **9**

1 Project Overview

This document explains the complete environment setup procedure for the ESP32 CSI Intrusion Detection project.

The project uses:

- ESP32 CSI passive sensing
- Python-based CSI parsing
- CNN+LSTM deep learning model
- TensorFlow GPU acceleration
- Jupyter Notebook development workflow
- Real-time CSI inference dashboard

2 Project Architecture

```
ESP32-E Passive Sniffer
        ↓
Serial CSI Stream
        ↓
Python CSI Parser
        ↓
Amplitude + Phase Feature Extraction
        ↓
Sliding Window Creation
        ↓
CNN + LSTM Deep Learning Model
        ↓
Real-Time Intrusion Prediction
```

3 Linux / Ubuntu Environment Setup

3.1 Step 1: Create Project Directory

```
mkdir -p ~/esp_projects
cd ~/esp_projects
```

3.2 Step 2: Create Python Virtual Environment

```
python3.10 -m venv ~/esp_projects/csi_realtime_ml/csi_env
```

3.3 Step 3: Activate the Environment

```
source ~/esp_projects/csi_realtime_ml/csi_env/bin/activate
```

After activation, the terminal should show:

```
(csi_env)
```

3.4 Step 4: Upgrade pip

```
python -m pip install --upgrade pip
```

3.5 Step 5: Install Required Packages

3.5.1 Core Scientific Packages

```
pip install numpy pandas scipy matplotlib seaborn
```

3.5.2 Machine Learning Packages

```
pip install scikit-learn tensorflow keras
```

3.5.3 Serial Communication Packages

```
pip install pyserial
```

3.5.4 Notebook and Kernel Packages

```
pip install notebook jupyterlab ipykernel
```

3.5.5 Visualization Packages

```
pip install plotly
```

3.5.6 Install All Packages Together

```
pip install \
numpy \
pandas \
scipy \
matplotlib \
seaborn \
scikit-learn \
tensorflow \
keras \
```

```
pyserial \
notebook \
jupyterlab \
ipykernel \
plotly
```

4 Windows Environment Setup

4.1 Step 1: Create Environment

```
python -m venv csi_env
```

4.2 Step 2: Activate Environment

```
.\csi_env\Scripts\activate
```

4.3 Step 3: Install Packages

```
pip install numpy pandas scipy matplotlib seaborn
pip install scikit-learn tensorflow keras
pip install pyserial notebook jupyterlab ipykernel
pip install plotly
```

4.4 Step 4: Register Jupyter Kernel

```
python -m ipykernel install \
--user \
--name csi_env \
--display-name "Python␣(csi_env)"
```

5 Registering the Jupyter Kernel

5.1 Linux Kernel Registration

```
source ~/esp_projects/csi_realtime_ml/csi_env/bin/activate
```

```
python -m ipykernel install \
--user \
--name csi_env \
--display-name "Python␣(csi_env)"
```

5.2 Verify Kernel Installation

```
jupyter kernelspec list
```

Expected output:

```
csi_env
```

6 Opening the Project in VS Code

6.1 Open VS Code

```
cd ~/esp_projects/csi_intrusion_pipeline
code .
```

6.2 Open Notebook

Open:

```
Notebook/Train and test.ipynb
```

6.3 Select Kernel

Inside the notebook:

```
Top Right -> Select Kernel
```

Choose:

```
Python (csi_env)
```

6.4 If Kernel Does Not Appear

Reload VS Code:

```
Ctrl + Shift + P
Developer: Reload Window
```

Then select:

```
Python (csi_env)
```

7 Testing the Environment

Run the following notebook cell:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
import serial
import tensorflow as tf

print("NumPy:", np.__version__)
print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

Expected output:

```
GPU: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU')]
```

8 TensorFlow GPU Verification

8.1 Check NVIDIA GPU

```
nvidia-smi
```

8.2 Check TensorFlow GPU Access

```
python -c "import␣tensorflow␣as␣tf;␣\
print(tf.config.list_physical_devices('GPU'))"
```

9 Package Summary

Package	**Purpose**
numpy	Numerical computation and array processing
pandas	CSV and dataframe handling
scipy	Signal processing utilities
matplotlib	Visualization and live plotting
seaborn	Advanced statistical visualization
scikit-learn	Machine learning preprocessing and evaluation
tensorflow	Deep learning framework
keras	CNN+LSTM model API
pyserial	ESP32 serial communication
notebook	Jupyter notebook support

jupyterlab	Interactive notebook interface
ipykernel	Jupyter kernel registration
plotly	Interactive visualization support

10 Running the Notebook

10.1 Launch Jupyter Notebook

```
jupyter notebook
```

or

```
jupyter lab
```

11 Real-Time CSI Dashboard Execution

11.1 Activate Environment

```
source ~/esp_projects/csi_realtime_ml/csi_env/bin/activate
```

11.2 Run Live Dashboard

```
cd ~/esp_projects/csi_intrusion_pipeline/realtime

python live_dashboard.py
```

12 Common Problems and Fixes

12.1 Problem: No module named numpy

This means the notebook is not using the correct kernel.

Fix:

```
Top Right -> Select Python (csi_env)
```

12.2 Problem: Python (csi_env) does not appear

Run:

```
source ~/esp_projects/csi_realtime_ml/csi_env/bin/activate

python -m ipykernel install \
--user \
--name csi_env \
--display-name "Python␣(csi_env)"
```

Then reload VS Code.

12.3 Problem: No module named serial

Install pyserial:

```
pip install pyserial
```

12.4 Problem: TensorFlow GPU not detected

Check GPU:

```
nvidia-smi
```

Then check TensorFlow:

```
python -c "import␣tensorflow␣as␣tf;␣\
print(tf.config.list_physical_devices('GPU'))"
```

13 Final Recommended Kernel

`Python (csi_env)`

for:

- CSI parsing
- Feature extraction
- CNN+LSTM model training
- Model testing
- Real-time CSI inference dashboard